

Software Architecture

SE6103

Sachin Tharaka
(BSc Honours in Software Engineering)

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: Introduction to Software Architectural pattern
- Topic 03: ADL
- Topic 04: 4+1 Architecture
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: Introduction to Software Architectural pattern
- Topic 03: ADL
- **Topic 04: 4+1 Architecture**
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

4+1 Architecture Model

From Lecture 2

Stakeholders in Software Architecture

Architecture serves multiple stakeholders

- Developers
- Project managers
- System administrators
- Business owners
- Clients
- End users



Different PoVs as **Developer** cares about modularity. **Businesses** care about cost and scalability.

sachin@foc.sab.ac.lk

Department of Software Engineering

SE6103

14

Multiple Perspectives in Architecture

- Logical view (functional elements)
- Development view (module organization)
- Process view (runtime behavior)
- Physical view (deployment)



sachin@foc.sab.ac.lk

Department of Software Engineering

SE6103

15

What is Software Architecture?

- High-level structure of a system
- Defines components and interactions
- Focuses on design decisions
- Impacts performance, scalability, maintainability

Why Multiple Views?

- Different stakeholders have different concerns
- Single diagram is not sufficient
- Helps manage system complexity

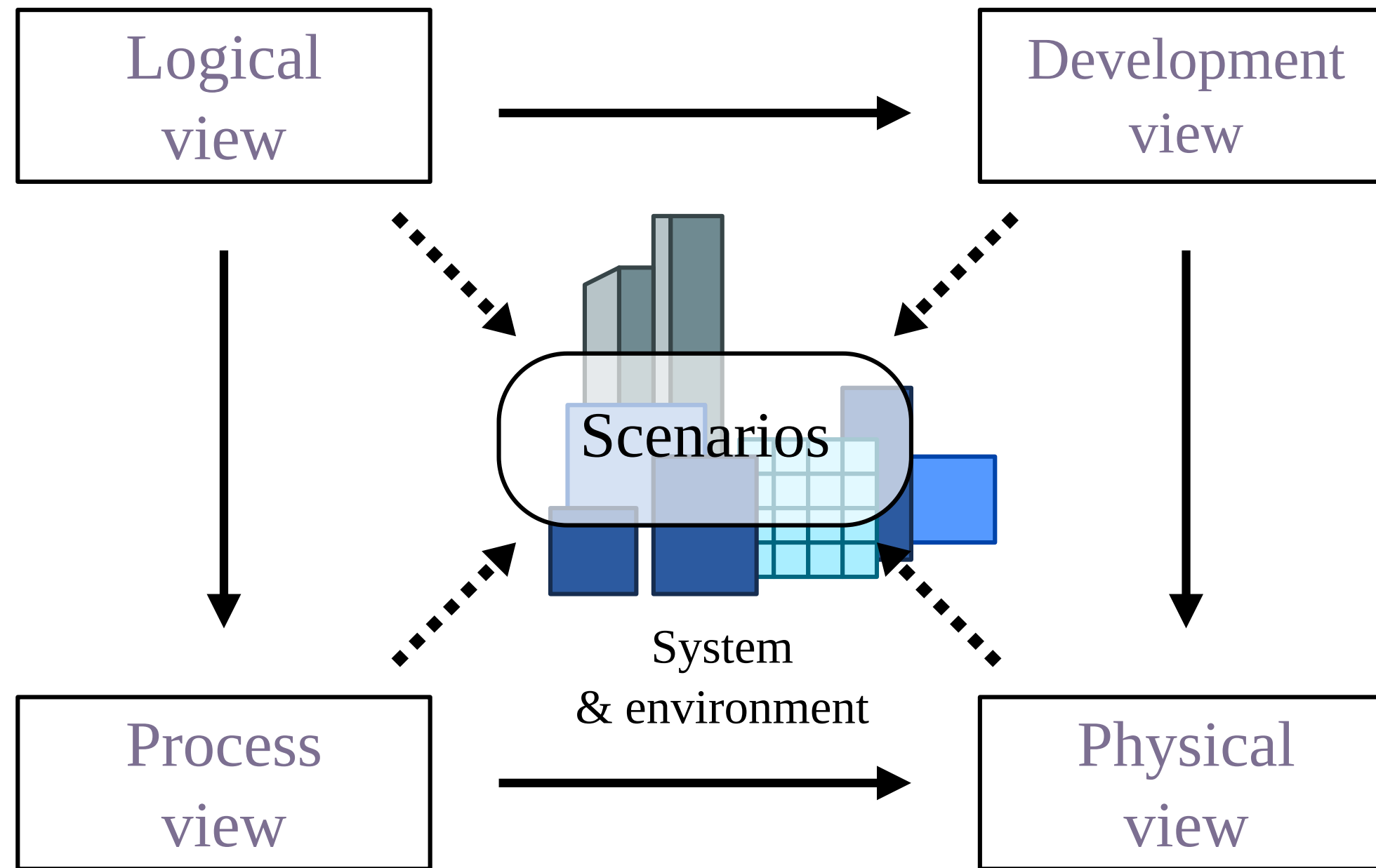
Examples:

- Developers → code structure
- Users → functionality
- DevOps → deployment

Introduction to 4+1 Model

- Proposed by Philippe Kruchten
- Uses 5 views to describe architecture
- Combines structure + behavior

Overview of 4+1 Views



Source: Wikipedia

Logical View – Overview

Aspect	Description
Focus	Functional requirements
Purpose	Describe what the system does
Perspective	End-user and analyst
Approach	Object-oriented modeling
Key Idea	Captures system functionality

Logical View – Details

Element	Description
Classes	Represent system entities
Objects	Instances of classes
Relationships	Associations, inheritance, dependencies
Business Logic	Core system functionality
UML Diagrams	Class diagrams, object diagrams

Development View – Overview

Aspect	Description
Focus	Code structure
Purpose	Organize software modules
Perspective	Developer/programmer view
Key Idea	How the system is built

Development View – Details

Element	Description
Modules	Independent code units
Packages	Group of related modules
Components	Reusable system parts
Dependencies	Relationships between modules
UML Diagrams	Package diagrams, component diagrams

Process View – Overview

Aspect	Description
Focus	Runtime behavior
Purpose	Handle concurrency and performance
Perspective	System engineer view
Key Idea	System execution flow

Process View – Details

Element	Description
Processes	Independent execution units
Threads	Lightweight execution paths
Communication	Interaction between processes
Synchronization	Managing shared resources
UML Diagrams	Sequence diagrams, activity diagrams

Physical View – Overview

Aspect	Description
Focus	Deployment environment
Purpose	Map software to hardware
Perspective	DevOps / infrastructure view
Key Idea	Where the system runs

Physical View – Details

Element	Description
Servers	Machines hosting applications
Nodes	Physical or virtual devices
Network Topology	Communication structure
Distribution	System spread across locations
UML Diagrams	Deployment diagrams

Scenario View (+1) – Overview

Aspect	Description
Focus	Use cases / scenarios
Purpose	Validate architecture
Perspective	All stakeholders
Key Idea	Connects all 4 views
Example	User interactions with system

Elements in Scenario View

Element	Description
Use Cases	High-level user goals (e.g., Login, Place Order)
Scenarios	Step-by-step execution of a use case
Actors	Users or external systems interacting with the system
Interaction Flow	Sequence of actions between components
Events	Triggers that start or affect a scenario
Validation Cases	Scenarios used to verify architecture correctness

Key Idea in Scenario View

- **Scenario view is behavior-driven, not structure-driven**
- **It ties together all 4 views:**
 - Logical → involved classes
 - Development → modules used
 - Process → runtime flow
 - Physical → deployment nodes

Example (Login Scenario)

Step	Action
1	Actor enters username & password
2	System validates credentials
3	Authentication process runs
4	Database is accessed
5	Response returned to user

Activity: Design a System Using 4+1 Model

Selects **ONE** system:

- Online Shopping System
- University Management System
- Ride Sharing App
- Food Delivery System
- Library Management System

1. Logical View

- Identify:
 - Main classes/entities
 - Relationships

2. Development View

- Define:
 - Modules / packages
 - System components

3. Process View

- Describe:
 - Runtime behavior
 - Concurrency (if any)

4. Physical View

- Show:
 - Deployment (servers, DB, clients)

5. Scenario View (+1)

- Create:
 - 1-2 use cases
 - Step-by-step scenario

Deliverables

Each group should prepare

View	Expected Output
Logical	Simple class diagram
Development	Module/component breakdown
Process	Sequence or flow diagram
Physical	Deployment diagram
Scenario	Use case + steps

University Management System (4+1 Model)

Logical View (Functional Design)

Class	Description
Student	Represents a student (ID, name, program)
Lecturer	Represents lecturers
Course	Course details (code, name, credits)
Enrollment	Links students to courses
Department	Academic department
Exam	Exam details
Result	Stores student grades

Relationships

Student $\leftrightarrow$ Course $\rightarrow$ Many-to-Many (via Enrollment)

Lecturer $\rightarrow$ Course (One-to-Many)

Course $\rightarrow$ Department (Many-to-One)

Student $\rightarrow$ Result (One-to-Many)

University Management System (4+1 Model)

Development View (Implementation Structure)

Module	Responsibility
User Management	Handles login, roles (admin, student, lecturer)
Course Management	Manage courses and departments
Enrollment Module	Student course registration
Examination Module	Exams and grading
Notification Module	Emails/SMS alerts
Database Layer	Data persistence

Architecture Style

- Layered Architecture:
 - Presentation Layer (UI)
 - Business Logic Layer
 - Data Access Layer

University Management System (4+1 Model)

Process View (Runtime Behavior)

Process	Description
Authentication	User login validation
Course Registration	Handles concurrent enrollments
Result Processing	Calculates and stores grades
Notification Process	Sends alerts asynchronously

Concurrency

- Multiple students enrolling simultaneously
- Parallel result processing

Communication

- API calls between frontend and backend
- Database queries

University Management System (4+1 Model)

Physical View (Deployment)

Component	Description
Client Devices	Web/mobile apps used by students & staff
Web Server	Hosts frontend
Application Server	Runs backend logic
Database Server	Stores system data
Network	Wifi /campus network

Deployment Architecture

- 3-tier architecture:
- Client → Server → Database

Optional:

- Cloud deployment (AWS/Azure)

University Management System (4+1 Model)

Scenario: Student Course Registration

Step	Action
1	Student logs into system
2	System authenticates user
3	Student selects courses
4	System checks prerequisites
5	Enrollment is recorded
6	Confirmation notification sent

4+1 Architecture Model

Thank You!